

POSTGRESQL 18

Database Upgrade & Patching

Technical Standard Operating Procedure (SOP)

Document Type	Technical SOP / Upgrade Runbook
Lab Platform	Ubuntu 26.04
Upgrade Path	PostgreSQL 16.15 → PostgreSQL 18.6
Database	dvdrental
Primary Method	pg_upgrade – COPY Mode
Draft from	Praveen Madupu
Date	31 August 2026 – Monday

Purpose: Controlled major upgrade, validation, rollback and minor patching guidance.

Document Control

Document Status	Final – Tested & Verified Edition
Technical Scope	PostgreSQL 16.15 → 18.6; COPY mode; LINK/CLONE notes; minor patching
Operating System	Ubuntu 26.04
Source Basis	Tested lab observations and verified PostgreSQL 18 technical procedure
Validation Basis	PostgreSQL 18 documented behavior and lab-specific checkpoints
Change Principle	Stop on failed compatibility checks; do not bypass pg_upgrade errors

Scope and Safety Statement

This SOP is written for the documented lab topology. Production execution must use the actual server paths, package repository, extensions, tablespaces, backup architecture, HA/replication design, and approved change process. If `pg_upgrade --check` reports an error, stop and correct the reported condition before continuing.

Table of Contents

Resolved page numbers are included below for immediate use. Page numbers refer to this document edition.

No.	Section	Page
1	Database Upgrade and Patching – Basics	5
2	PostgreSQL Version Numbering	5
3	Major vs Minor Version Upgrade	6
4	Why Database Upgrades Are Required	6
5	PostgreSQL Major Upgrade Methods	7
6	`pg_upgrade` Utility	8
7	PostgreSQL 16 to 18 – Upgrade Using Copy Mode	9
8	Load / Validate Sample Database	9
9	Major Upgrade Pre-Checks	10
10	CPU, Memory and Capacity	11
11	PostgreSQL 18 Installation and Initialization	11
12	Data Checksum Compatibility	12
13	Configure Temporary PostgreSQL 18 Port	12
14	Configuration Migration	13
15	Extension Pre-Check	13
16	`pg_upgrade --check`	13
17	Production Upgrade Preparation	14
18	Capture Tested Baseline Information	15
19	Stop PostgreSQL 16	16
20	Actual PostgreSQL 16 → 18 Upgrade – Copy Mode	17
21	PostgreSQL 18 `pg_upgrade` Statistics Improvement	17
22	Configure Production Port	17
23	Start PostgreSQL 18	18
24	Validate DVD Rental Database	18
25	Additional Post-Upgrade Validation	19
26	Statistics Maintenance	20
27	Application Handover	20
28	Rollback – Copy Mode	21
29	Critical Rollback Point – New Production Writes	21

No.	Section	Page
30	PostgreSQL 18 → 16 Logical Downgrade Warning	22
31	Clean Up PostgreSQL 16	22
32	PostgreSQL 16 → 18 Link Mode Upgrade	22
33	Linux Inode Basics	23
34	Why Link Mode Uses Hard Links	23
35	Link Mode Filesystem Requirement	23
36	Link Mode Upgrade Preparation	24
37	Stop Application and PostgreSQL 16	24
38	Perform Link Mode Upgrade	24
39	Configure PostgreSQL 18	24
40	Start PostgreSQL 18	25
41	Critical Link-Mode Rollback Rule	25
42	Link Mode Rollback Window	25
43	PostgreSQL 18 Clone Mode	25
44	Copy vs Link vs Clone	26
45	Minor Version Upgrade / Patching	26
46	PostgreSQL 18 Minor-Version Example	26
47	Ubuntu Package Validation	27
48	PostgreSQL 18 Minor Upgrade Preparation	27
49	PostgreSQL 18.x → 18.6 Minor Upgrade	27
50	Start PostgreSQL After Minor Upgrade	27
51	Minor Upgrade Production Procedure	28
52	Minor-Release Release-Notes Rule	28
53	Minor-Version Rollback	28
54	Controlled Minor-Version Package Rollback	29
55	Major Upgrade vs Minor Upgrade	29
56	Major Upgrade Options	30
57	`pg_upgrade` Production Checklist	30
58	Additional Useful Commands	30
59	Backup Validation Before Upgrade	31
60	Do Not Forget Replication	32
61	Do Not Delete the Old Cluster Immediately	32
62	Validated Technical Rules	33
63	Production Recommendation – PostgreSQL 16 → 18	34
64	Quick Reference Cheat Sheet	34
65	Final DBA Takeaways	36
66	Official PostgreSQL References	38
67	Summary	38

1. Database Upgrade and Patching – Basics

Upgrade

An **upgrade** means moving PostgreSQL from one version to another version.

Typical reasons include:

- Getting new PostgreSQL features.
- Moving from an older supported version to a newer supported version.
- Maintaining application compatibility.
- Improving performance or manageability.
- Moving away from an End-of-Life PostgreSQL version.

Example:

```
PostgreSQL 16 → PostgreSQL 18
```

This is a **major version upgrade**.

Patching

Patching generally means applying a PostgreSQL maintenance release containing:

- Security fixes.
- Bug fixes.
- Data corruption fixes.
- Stability fixes.
- Low-risk improvements.

PostgreSQL maintenance fixes are delivered through **minor releases**.

Example:

```
PostgreSQL 18.4 → PostgreSQL 18.6
```

This is a **minor version update**.

2. PostgreSQL Version Numbering

Starting with PostgreSQL 10, PostgreSQL uses:

```
Major.Minor
```

Examples:

```
16.15
17.11
18.6
```

For:

```
18.6
```

the meaning is:

```
18 = Major Version
6  = Minor Version
```

Therefore:

16 → 18

is a **major upgrade**.

Whereas:

18.4 → 18.6

is a **minor version update**.

Before PostgreSQL 10, the numbering worked differently.

Example:

9.5.3

means:

9.5 = Major Version
3 = Minor Release

3. Major vs Minor Version Upgrade

Upgrade Type	Example	Data Directory Reusable Directly?	Typical Method
Major Upgrade	16 → 18	No	`pg_upgrade`, dump/restore, logical replication
Minor Upgrade	18.4 → 18.6	Yes	Update binaries/packages and restart
Security/Bug Fix	Delivered through minor release	Yes	Minor version update

A PostgreSQL major release can change:

- System catalogs.
- Internal metadata.
- SQL behavior.
- Configuration parameters.
- Extension interfaces.
- Internal structures.

Therefore PostgreSQL 16 and PostgreSQL 18 cannot simply reuse the same cluster directory without a supported major-upgrade mechanism.

For minor releases, PostgreSQL maintains physical storage compatibility within the same major release.

4. Why Database Upgrades Are Required

Database upgrades may be required for:

New Features

PostgreSQL 18 introduced enhancements including:

- Asynchronous I/O.
- UUIDv7 support.
- B-tree skip scans.
- OAuth authentication support.
- Improved `pg\_upgrade`.
- Additional SQL and monitoring enhancements.

Security Fixes

Minor releases regularly contain security fixes.

Bug Fixes

Minor releases can contain fixes for:

- Crashes.
- Incorrect query results.
- Replication issues.
- WAL issues.
- Data corruption risks.
- Planner issues.

End of Life

PostgreSQL major versions are supported for approximately five years.

Application Compatibility

Applications may require a newer PostgreSQL major release.

Infrastructure Compatibility

Operating-system or platform changes may require newer PostgreSQL packages.

5. PostgreSQL Major Upgrade Methods

PostgreSQL major-version migrations commonly use:

Method 1 – Logical Dump and Restore

Utilities:

```
pg_dump
pg_dumpall
pg_restore
psql
```

Architecture:

```
PostgreSQL 16
|
| logical export
v
Dump Files
|
| restore
v
PostgreSQL 18
```

Advantages:

- Simple concept.
- Clean logical rebuild.
- Can support platform changes.
- Useful for smaller environments.

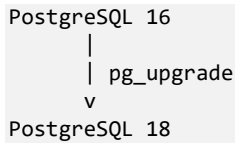
Disadvantages:

- Can take significant time.
- Requires dump storage.

- Downtime can be high for large databases.

Method 2 – `pg\_upgrade`

Architecture:



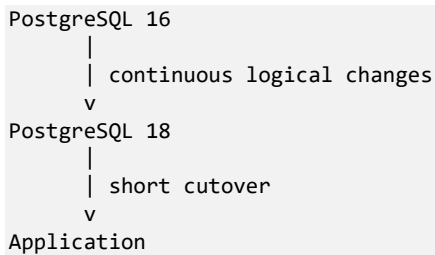
Advantages:

- Much faster than full logical dump/restore for large databases.
- Reuses existing user relation files through copy/link/clone/swap methods.
- Good for same-host or same-platform major upgrades.

Method 3 – Logical Replication

Useful where very low application downtime is required.

Architecture:



This requires separate planning for sequences, DDL, large objects, replication slots, cutover, rollback and application behavior.

6. `pg\_upgrade` Utility

`pg_upgrade` was officially introduced with **PostgreSQL 9.0**.

For a PostgreSQL 16 → 18 upgrade, always use the **new PostgreSQL version's `pg\_upgrade` binary**:

```
/usr/lib/postgresql/18/bin/pg_upgrade
```

`pg_upgrade` avoids logically exporting and restoring every row.

PostgreSQL 18 `pg_upgrade` supports multiple transfer modes, including:

```
copy
link
clone
copy_file_range
swap
```

For this lab, we use the default:

```
COPY MODE
```

7. PostgreSQL 16 to 18 – Upgrade Using Copy Mode

Tested Lab Environment

```

Operating System : Ubuntu 26.04
Host             : pgdb01

Source Version   : PostgreSQL 16.15
Source Cluster   : 16/main
Source Port      : 5432
Source Status    : ONLINE

Target Version   : PostgreSQL 18.6
Target Cluster   : 18/main
Preparation Port : 5433
Target Status    : DOWN

Database         : dvdrental
Upgrade Method    : pg_upgrade
Transfer Mode     : COPY

```

Important Ubuntu paths:

```

PG16 Binary Directory
/usr/lib/postgresql/16/bin

PG18 Binary Directory
/usr/lib/postgresql/18/bin

PG16 Data Directory
/var/lib/postgresql/16/main

PG18 Data Directory
/var/lib/postgresql/18/main

PG16 Configuration Directory
/etc/postgresql/16/main

PG18 Configuration Directory
/etc/postgresql/18/main

```

8. Load / Validate Sample Database

The lab uses the dvdrental database.

If it is already loaded, simply validate it:

```
sudo -u postgres psql -p 5432 -d dvdrental
```

List tables:

```
\dt
```

List views:

```
\dv
```

The tested environment already contains the DVD Rental objects and baseline row counts shown later in these notes.

9. Major Upgrade Pre-Checks

Check clusters:

```
pg_lsclusters
```

Expected preparation state:

```
16 main 5432 online
18 main 5433 down
```

Check versions:

```
/usr/lib/postgresql/16/bin/postgres --version
/usr/lib/postgresql/18/bin/postgres --version
```

Expected:

```
postgres (PostgreSQL) 16.15
postgres (PostgreSQL) 18.6
```

Check source data directory:

```
sudo -u postgres psql -p 5432 -Atc "SHOW data_directory;"
```

Expected:

```
/var/lib/postgresql/16/main
```

Check source port:

```
sudo -u postgres psql -p 5432 -Atc "SHOW port;"
```

Expected:

```
5432
```

Check disk:

```
df -h /
```

Check inode availability:

```
df -i /
```

Check source cluster size:

```
sudo du -sh /var/lib/postgresql/16/main
```

Check tablespaces:

```
sudo -u postgres psql -p 5432 -c "
SELECT
    spcname,
    pg_tablespace_location(oid)
FROM pg_tablespace;
"
```

COPY mode requires enough storage for the new cluster and copied relation files plus WAL, temporary files and operational headroom.

10. CPU, Memory and Capacity

Check memory:

```
free -h
```

Check CPU:

```
lscpu
```

Check CPU count:

```
nproc
```

The old and new production clusters are not expected to serve normal production traffic simultaneously during the actual `pg_upgrade`, but sufficient CPU, memory and I/O capacity are still required for:

- Compatibility checks.
- Temporary server startup.
- File transfer.
- Statistics regeneration.
- Post-upgrade validation.

11. PostgreSQL 18 Installation and Initialization

Install PostgreSQL 18 packages from the configured PostgreSQL repository:

```
sudo apt update
sudo apt install postgresql-18 postgresql-client-18 postgresql-contrib-18
```

Validate:

```
/usr/lib/postgresql/18/bin/postgres --version
```

Check clusters:

```
pg_lsclusters
```

For this tested lab, PostgreSQL 18 is already prepared as cluster:

```
18/main
```

on temporary port:

```
5433
```

The PostgreSQL 18 cluster must be initialized before `pg_upgrade`, but it does **not** need to remain started.

12. Data Checksum Compatibility

PostgreSQL 18 enables data checksums by default for newly initialized clusters.

Check PostgreSQL 16:

```
sudo -u postgres psql -p 5432 -Atc "SHOW data_checksums;"
```

Tested result:

```
off
```

Check PostgreSQL 18:

```
sudo -u postgres \
/usr/lib/postgresql/18/bin/pg_controldata \
/var/lib/postgresql/18/main \
| grep -i "Data page checksum"
```

For this lab, the target was initialized to match the source:

```
PG16 checksums = OFF
PG18 checksums = OFF
```

If a PostgreSQL 18 cluster must be recreated to match a non-checksum source, initialize it with:

```
sudo -u postgres /usr/lib/postgresql/18/bin/initdb \
-D /var/lib/postgresql/18/main \
--no-data-checksums
```

On Ubuntu/Debian, use the normal cluster-management tooling and permissions for your environment when recreating a cluster.

Do **not** disable checksums automatically in every environment. Match the source cluster's checksum state.

13. Configure Temporary PostgreSQL 18 Port

During preparation:

```
PG16 = 5432
PG18 = 5433
```

PostgreSQL 18 configuration file:

```
/etc/postgresql/18/main/postgresql.conf
```

Set:

```
port = 5433
```

Check:

```
grep -E '^[[:space:]]*port[[:space:]]*=' \
/etc/postgresql/18/main/postgresql.conf
```

During the actual pg_upgrade, both normal cluster instances are stopped and pg_upgrade temporarily starts the clusters itself.

14. Configuration Migration

Do **not** blindly copy PostgreSQL 16's complete configuration over PostgreSQL 18.

Compare and review:

```
diff -u \
/etc/postgresql/16/main/postgresql.conf \
/etc/postgresql/18/main/postgresql.conf
```

Review important parameters such as:

```
listen_addresses
port
max_connections
shared_buffers
```

```
work_mem
maintenance_work_mem
wal_level
max_wal_senders
max_replication_slots
archive_mode
archive_command
shared_preload_libraries
ssl
logging settings
```

Also review:

```
pg_hba.conf
pg_ident.conf
postgresql.auto.conf
include
include_if_exists
include_dir
```

Migrate only settings that are still valid and appropriate for PostgreSQL 18.

15. Extension Pre-Check

Check installed extensions:

```
sudo -u postgres psql -p 5432 -d dvdrental -c "
SELECT
    extname,
    extversion
FROM pg_extension
ORDER BY extname;
"
```

If third-party extensions exist, install their PostgreSQL 18-compatible binaries/libraries before running pg_upgrade.

Do not copy PostgreSQL 16 extension .so files into PostgreSQL 18.

16. `pg\_upgrade --check`

Create a working directory:

```
sudo mkdir -p /var/lib/postgresql/pg_upgrade_work
sudo chown postgres:postgres /var/lib/postgresql/pg_upgrade_work
sudo chmod 700 /var/lib/postgresql/pg_upgrade_work
```

Check cluster state:

```
pg_lsclusters
```

Expected:

```
16 main 5432 online
18 main 5433 down
```

Run:

```
sudo -u postgres bash -c '
cd /var/lib/postgresql/pg_upgrade_work

/usr/lib/postgresql/18/bin/pg_upgrade \
--old-bindir=/usr/lib/postgresql/16/bin \
```

```
--new-bindir=/usr/lib/postgresql/18/bin \
--old-datadir=/etc/postgresql/16/main \
--new-datadir=/etc/postgresql/18/main \
--old-options="-D /var/lib/postgresql/16/main" \
--new-options="-D /var/lib/postgresql/18/main" \
--old-port=5432 \
--new-port=5433 \
--username=postgres \
--check
,
```

For PostgreSQL 18, `-d/--old-datadir` and `-D/--new-datadir` refer to the old/new **cluster configuration directories**. Ubuntu/Debian stores configuration separately from the physical data directory, so the physical PGDATA paths are supplied to the temporary servers using the additional server options.

Expected result:

```
Clusters are compatible
```

If compatibility checking fails:

```
STOP
↓
Read the exact pg_upgrade error
↓
Correct the problem
↓
Run --check again
```

Do not continue until the check succeeds.

17. Production Upgrade Preparation

Before the production change:

```
Change approved
Maintenance window approved
Application downtime approved
Backup completed
Backup verification completed
Rollback plan documented
pg_upgrade --check successful
Extension compatibility confirmed
Disk space confirmed
Tablespaces reviewed
Checksum state confirmed
```

Stop or suspend:

```
Application services
Batch jobs
ETL jobs
Scheduled writers
Write-based reports
Database maintenance jobs
```

Check remaining sessions:

```
sudo -u postgres psql -p 5432 -c "
SELECT
    pid,
    username,
    datname,
    application_name,
```

```

    client_addr,
    state
FROM pg_stat_activity
WHERE pid <> pg_backend_pid();
"

```

18. Capture Tested Baseline Information

Tested Row Counts

actor	200
customer	599
film	1000
inventory	4581
rental	16044
payment	14596

Capture:

```

sudo -u postgres psql -p 5432 -d dvdrental -c "
SELECT 'actor' AS table_name, COUNT(*) FROM actor
UNION ALL
SELECT 'customer', COUNT(*) FROM customer
UNION ALL
SELECT 'film', COUNT(*) FROM film
UNION ALL
SELECT 'inventory', COUNT(*) FROM inventory
UNION ALL
SELECT 'rental', COUNT(*) FROM rental
UNION ALL
SELECT 'payment', COUNT(*) FROM payment;
"

```

Tested Object Counts

Tables	15
Views	7
Indexes	32
Sequences	13

Capture:

```

sudo -u postgres psql -p 5432 -d dvdrental -c "
SELECT 'Tables' AS object_type, COUNT(*) AS object_count
FROM pg_tables
WHERE schemaname='public'

UNION ALL

SELECT 'Views', COUNT(*)
FROM pg_views
WHERE schemaname='public'

UNION ALL

SELECT 'Indexes', COUNT(*)
FROM pg_indexes
WHERE schemaname='public'

UNION ALL

```

```
SELECT 'Sequences', COUNT(*)
FROM pg_sequences
WHERE schemaname='public';
"
```

These are the tested lab values to compare after upgrade.

19. Stop PostgreSQL 16

After application traffic is stopped:

```
sudo pg_ctlcluster 16 main stop
```

Make sure PostgreSQL 18 is also down:

```
sudo pg_ctlcluster 18 main stop
```

Validate:

```
pg_lsclusters
```

Required:

```
16 main 5432 down
18 main 5433 down
```

20. Actual PostgreSQL 16 → 18 Upgrade – Copy Mode

Use the same command that passed --check, removing only --check:

```
sudo -u postgres bash -c '
cd /var/lib/postgresql/pg_upgrade_work

/usr/lib/postgresql/18/bin/pg_upgrade \
--old-bindir=/usr/lib/postgresql/16/bin \
--new-bindir=/usr/lib/postgresql/18/bin \
--old-datadir=/etc/postgresql/16/main \
--new-datadir=/etc/postgresql/18/main \
--old-options="-D /var/lib/postgresql/16/main" \
--new-options="-D /var/lib/postgresql/18/main" \
--old-port=5432 \
--new-port=5433 \
--username=postgres
'
```

No --link option is specified.

Therefore this uses:

```
COPY MODE
```

Successful completion should end with an upgrade-complete message.

Do not delete PostgreSQL 16.

21. PostgreSQL 18 `pg\_upgrade` Statistics Improvement

PostgreSQL 18 `pg_upgrade` transfers **most optimizer statistics** by default.

It does not transfer everything.

Examples that may need regeneration include:

```
Extended statistics created with CREATE STATISTICS
Extension-defined statistics
Cumulative statistics
```

PostgreSQL therefore recommends rebuilding missing and cumulative statistics after the upgrade.

22. Configure Production Port

PostgreSQL 16 used:

```
5432
```

PostgreSQL 18 used temporary preparation port:

```
5433
```

After PG16 is stopped, configure PG18 to use:

```
5432
```

Edit:

```
sudo nano /etc/postgresql/18/main/postgresql.conf
```

Set:

```
port = 5432
```

Verify:

```
grep -E '^[[:space:]]*port[[:space:]]*=' \
/etc/postgresql/18/main/postgresql.conf
```

23. Start PostgreSQL 18

Start:

```
sudo pg_ctlcluster 18 main start
```

Validate:

```
pg_lsclusters
```

Expected:

```
16 main 5432 down
18 main 5432 online
```

Check listener:

```
sudo ss -lntp | grep ':5432'
```

Check version:

```
sudo -u postgres psql -p 5432 -c "SELECT version();"
```

Check:

```
sudo -u postgres psql -p 5432 -c "SHOW server_version;"
```


24. Validate DVD Rental Database

Connect:

```
sudo -u postgres psql -p 5432 -d dvdrental
```

Check:

```
SELECT current_database();
SELECT version();
```

List tables:

```
\dt
```

List views:

```
\dv
```

25. Additional Post-Upgrade Validation

Re-run tested row counts:

```
SELECT 'actor' AS table_name, COUNT(*) FROM actor
UNION ALL
SELECT 'customer', COUNT(*) FROM customer
UNION ALL
SELECT 'film', COUNT(*) FROM film
UNION ALL
SELECT 'inventory', COUNT(*) FROM inventory
UNION ALL
SELECT 'rental', COUNT(*) FROM rental
UNION ALL
SELECT 'payment', COUNT(*) FROM payment;
```

Expected:

actor	200
customer	599
film	1000
inventory	4581
rental	16044
payment	14596

Re-run object counts:

```
SELECT 'Tables' AS object_type, COUNT(*) AS object_count
FROM pg_tables
WHERE schemaname='public'

UNION ALL

SELECT 'Views', COUNT(*)
FROM pg_views
WHERE schemaname='public'

UNION ALL

SELECT 'Indexes', COUNT(*)
FROM pg_indexes
WHERE schemaname='public'
```

```
UNION ALL

SELECT 'Sequences', COUNT(*)
FROM pg_sequences
WHERE schemaname='public';
```

Expected:

```
Tables      15
Views       7
Indexes     32
Sequences   13
```

Check invalid indexes:

```
SELECT
    n.nspname,
    c.relname
FROM pg_index i
JOIN pg_class c
    ON c.oid = i.indexrelid
JOIN pg_namespace n
    ON n.oid = c.relnamespace
WHERE NOT i.indisvalid;
```

Check extensions:

```
SELECT
    extname,
    extversion
FROM pg_extension
ORDER BY 1;
```

Review logs:

```
sudo tail -100 /var/log/postgresql/postgresql-18-main.log
```

26. Statistics Maintenance

First generate statistics where missing:

```
sudo -u postgres \
/usr/lib/postgresql/18/bin/vacuumdb \
--all \
--analyze-in-stages \
--missing-stats-only
```

Then refresh cumulative statistics:

```
sudo -u postgres \
/usr/lib/postgresql/18/bin/vacuumdb \
--all \
--analyze-only
```

For large production environments, parallel jobs can be considered after reviewing CPU and I/O capacity:

```
sudo -u postgres \
/usr/lib/postgresql/18/bin/vacuumdb \
--all \
--analyze-only \
--jobs=4
```

27. Application Handover

Once DBA validation succeeds, ask the application team to start services and test:

```
Login
SELECT
INSERT
UPDATE
DELETE
Functions
Batch jobs
Reports
APIs
Connection pooling
```

Monitor:

```
PostgreSQL logs
CPU
Memory
Disk I/O
Connections
Locks
Long-running queries
Application errors
Replication lag
Backup jobs
Monitoring alerts
```

28. Rollback – Copy Mode

COPY mode keeps PostgreSQL 16 relation files independent from PostgreSQL 18.

If PG18 validation fails **before new production writes are accepted**:

Stop PG18:

```
sudo pg_ctlcluster 18 main stop
```

Restore PG16 endpoint/configuration if needed.

Start PG16:

```
sudo pg_ctlcluster 16 main start
```

Validate:

```
pg_lsclusters
```

Connect:

```
sudo -u postgres psql -p 5432 -d dvdrental
```

This is why COPY mode provides the simplest immediate rollback position.

29. Critical Rollback Point – New Production Writes

Once PostgreSQL 18 accepts new production writes:

```
PG16 = data at original cutover
PG18 = cutover data + new production transactions
```

PostgreSQL 16 is now stale.

Do not simply stop PG18 and restart PG16.

A rollback after new PG18 writes requires a separately designed data-reconciliation/recovery strategy.

30. PostgreSQL 18 → 16 Logical Downgrade Warning

Do not assume:

```
PG18 pg_dump
      ↓
Restore to PG16
```

is a guaranteed rollback method.

Logical dumps from newer PostgreSQL versions are not guaranteed to restore successfully into older major versions.

PostgreSQL 18 may use:

```
New syntax
New data types
New object definitions
New features
```

that PostgreSQL 16 does not understand.

A PG18 → PG16 downgrade must therefore be separately designed and tested.

31. Clean Up PostgreSQL 16

Do not remove PostgreSQL 16 immediately.

Wait until:

```
DBA validation complete
Application validation complete
Business validation complete
Performance acceptable
PG18 backup successful
Restore capability confirmed
Monitoring stable
Replication healthy
Rollback window closed
Change approved for closure
```

Review the cleanup instructions generated by `pg_upgrade`.

Only then remove the old cluster and, when no longer needed, the PostgreSQL 16 packages.

32. PostgreSQL 16 → 18 Link Mode Upgrade

Link mode uses:

```
--link
```

or:

```
-k
```

Example uses the same Ubuntu split configuration/data-directory layout:

```
sudo -u postgres bash -c '
cd /var/lib/postgresql/pg_upgrade_work

/usr/lib/postgresql/18/bin/pg_upgrade \
--old-bindir=/usr/lib/postgresql/16/bin \
--new-bindir=/usr/lib/postgresql/18/bin \
--old-datadir=/etc/postgresql/16/main \
--new-datadir=/etc/postgresql/18/main \
--old-options="-D /var/lib/postgresql/16/main" \
--new-options="-D /var/lib/postgresql/18/main" \
--old-port=5432 \
--new-port=5433 \
--username=postgres \
--link
'
```

Advantages:

```
Very fast
Very low additional data-file storage
```

Main disadvantage:

```
Rollback becomes highly restricted
```

33. Linux Inode Basics

An inode is **not the physical file**.

An inode is filesystem metadata describing a filesystem object.

It contains information such as:

```
File type
Owner
Permissions
Size
Timestamps
Link count
Pointers to data blocks
```

Display inode numbers:

```
ls -li
```

34. Why Link Mode Uses Hard Links

A hard link creates another directory entry pointing to the same inode/data blocks.

Therefore, with `pg_upgrade --link`, old and new relation filenames can refer to the same underlying data.

That is why link mode:

```
Uses very little additional storage
Runs very quickly
```

It is also why rollback becomes dangerous after PostgreSQL 18 starts modifying those shared files.

35. Link Mode Filesystem Requirement

Hard links cannot cross filesystems.

Therefore old and new relation files that must be linked need to be on compatible locations within the same filesystem.

User tablespaces require the same consideration.

36. Link Mode Upgrade Preparation

Before link mode:

```
Backup verified
Rollback strategy approved
PG18 binaries installed
PG18 cluster initialized
Checksum state matched
Extensions installed
pg_upgrade --check passed
Application downtime approved
Filesystem requirement validated
```

Run compatibility check first using the same --check command shown earlier.

37. Stop Application and PostgreSQL 16

Stop application traffic.

Then:

```
sudo pg_ctlcluster 16 main stop
sudo pg_ctlcluster 18 main stop
```

Validate:

```
pg_lsclusters
```

Both clusters must be down.

38. Perform Link Mode Upgrade

Run the command shown in Section 32 with:

```
--link
```

Pay close attention to the warning generated by pg_upgrade.

39. Configure PostgreSQL 18

Set the production port:

```
port = 5432
```

in:

```
/etc/postgresql/18/main/postgresql.conf
```

Complete configuration review before starting PostgreSQL 18.

40. Start PostgreSQL 18

Start:

```
sudo pg_ctlcluster 18 main start
```

Run statistics maintenance and validation as described in the COPY-mode procedure.

41. Critical Link-Mode Rollback Rule

Restoring only:

```
pg_control.old
```

after PG18 has already started is **not safe**.

With link mode, PostgreSQL 16 and PostgreSQL 18 can share user relation files.

Once PG18 starts and modifies those files:

```
DO NOT START THE ORIGINAL LINKED PG16 CLUSTER
```

Restoring pg_control.old does not make the shared relation files PostgreSQL-16-compatible again.

42. Link Mode Rollback Window

Before PostgreSQL 18 has ever been started, follow the exact rollback instructions generated by pg_upgrade.

After PostgreSQL 18 has started:

```
Do not restart the original linked PostgreSQL 16 cluster.
```

If strong rollback requirements exist, prefer:

```
COPY mode  
Filesystem/storage snapshot  
Clone mode where supported  
Separate physical backup  
Logical replication migration
```

depending on architecture.

43. PostgreSQL 18 Clone Mode

PostgreSQL also supports:

```
--clone
```

where supported by the filesystem.

Clone mode uses copy-on-write filesystem cloning.

Potential benefits:

```
Fast  
Low initial additional disk usage  
Old cluster remains independent
```

Filesystem support must be validated before choosing this mode.

44. Copy vs Link vs Clone

Mode	Data Transfer	Extra Space	Speed	Old Cluster After New Starts
Copy	Full copy	Higher	Slower	Independent
Link	Hard links	Very low	Very fast	Unsafe to restart
Clone	Copy-on-write	Low initially	Very fast	Independent
`copy_file_range`	Filesystem dependent	Depends	Fast	Independent

For this tested lab:

```
COPY MODE
```

is the preferred teaching and rollback-friendly method.

45. Minor Version Upgrade / Patching

PostgreSQL maintenance fixes are delivered through minor releases. Minor-version updating and patching are therefore handled by moving to a newer minor release within the same major version.

```
Typical reasons include security fixes.
Bug and data-corruption fixes.
Crash and stability fixes.
Replication and recovery fixes.
Query-correctness fixes.
Performance fixes.
Low-risk improvements to existing functionality.
```

Representative upgrade path:

```
18.2 → 18.4 → 18.6
```

A PostgreSQL 18.x minor update normally does not require:

```
pg_upgrade
a new PostgreSQL data directory
pg_dump / pg_restore as the upgrade mechanism
re-initializing the PostgreSQL 18 cluster
```

The existing PostgreSQL 18 data directory remains in place and is started with the newer PostgreSQL 18 binaries after the package update.

46. PostgreSQL 18 Minor-Version Example

The demonstrated maintenance pattern is PostgreSQL 18.2 → 18.4. The same operational model applies to a later PostgreSQL 18 minor release after reviewing all applicable release notes.

```
Demonstrated path: 18.2 → 18.4
Current verified path used in this SOP: 18.4 → 18.6
```

Both are minor-version updates within PostgreSQL major version 18.

The important rule is:

```
replace/update PostgreSQL 18 binaries/packages → keep the PostgreSQL 18 data directory → restart and
validate
```


47. Ubuntu Package Validation

Check the installed PostgreSQL 18 packages:

```
dpkg -l | grep '^ii' | grep 'postgresql-18'
```

Check the candidate package version available from the configured repository:

```
apt-cache policy postgresql-18
```

Refresh package metadata:

```
sudo apt update
```

Before the maintenance window, confirm the exact PostgreSQL 18 package version APT plans to install. This is the Ubuntu equivalent of validating the exact PostgreSQL 18.x RPM level before a YUM-based minor update.

48. PostgreSQL 18 Minor Upgrade Preparation

Before a PostgreSQL 18.x minor update:

```
Read the release notes for the target release.
Review every skipped PostgreSQL 18 minor release.
Take and verify the approved backup or snapshot.
Confirm repository availability and the exact target package version.
Obtain the approved maintenance/change window.
Coordinate with the application team and stop application traffic.
Confirm extension/package compatibility.
Prepare and test the rollback procedure.
```

Record the current server version and data directory before the change:

```
sudo -u postgres psql -p 5432 -c "SELECT version(); SHOW data_directory;"
```

49. PostgreSQL 18.x → 18.6 Minor Upgrade

Stop application traffic and confirm that new transactions are no longer reaching PostgreSQL.

For the controlled maintenance window, stop the PostgreSQL 18 cluster:

```
sudo pg_ctlcluster 18 main stop
```

Refresh package metadata and update the PostgreSQL 18 packages:

```
sudo apt update

sudo apt install --only-upgrade \
postgresql-18 \
postgresql-client-18 \
postgresql-contrib-18
```

This updates the PostgreSQL 18 software while retaining the existing PostgreSQL 18 data directory. Do not run initdb and do not create a new data directory for a normal 18.x → 18.y minor update.

50. Start PostgreSQL After Minor Upgrade

Start PostgreSQL 18 using the existing data directory:

```
sudo pg_ctlcluster 18 main start
```

Validate cluster status:

```
pg_lsclusters
```

Confirm the new PostgreSQL minor version:

```
sudo -u postgres psql -p 5432 -c "SELECT version();"
```

Confirm that the same PostgreSQL 18 data directory is in use:

```
sudo -u postgres psql -p 5432 -Atc "SHOW data_directory;"
```

Validate databases, application connectivity, extensions, logs, and critical transactions before returning the service to normal operation.

51. Minor Upgrade Production Procedure

Recommended production sequence:

1. Read the target and skipped minor-release notes.
2. Test the exact package update in non-production.
3. Confirm the installed and target PostgreSQL 18 package versions.
4. Take and verify the approved backup/snapshot.
5. Obtain change approval and application downtime.
6. Ask the application team to stop services/traffic.
7. Stop PostgreSQL 18 for the controlled outage.
8. Update the PostgreSQL 18 binaries/packages; keep the existing PostgreSQL 18 data directory.
9. Start PostgreSQL 18.
10. Confirm the target server version and data directory.
11. Review PostgreSQL and operating-system logs.
12. Validate databases, objects, extensions and application connectivity.
13. Perform any release-note-specific follow-up action.
14. Ask the application team to start services.
15. Monitor the database and application after handover.

52. Minor-Release Release-Notes Rule

The basic minor-update pattern is:

```
stop workload → update PostgreSQL 18 packages → start the same PostgreSQL 18 data directory → validate
```

However, release notes can require additional actions such as:

```
REINDEX or other index maintenance
Configuration changes
Security-related changes
Data or catalog maintenance
Extension maintenance
```

Always review every applicable minor-release note before production execution.

53. Minor-Version Rollback

PostgreSQL minor releases within the same major version are designed to maintain physical storage compatibility, which is why the same PostgreSQL 18 data directory is used across PostgreSQL 18 minor releases.

A package downgrade must still be treated as a controlled rollback, not as an automatic guarantee.

Before downgrading, determine whether the newer minor release involved:

- REINDEX or other release-specific maintenance
- Extension updates
- Configuration changes
- Security-related changes
- Other manual follow-up actions

A rollback plan should include:

- Exact previous PostgreSQL 18 package versions
- Repository/package availability
- Verified backup or snapshot
- Extension compatibility
- Release-note review
- Database and application validation

54. Controlled Minor-Version Package Rollback

Use package rollback only when:

- The exact previous PostgreSQL 18 packages are available.
- The applicable release notes have been reviewed.
- Extension compatibility is confirmed.
- No incompatible release-specific follow-up action prevents rollback.
- The rollback has been tested in a representative non-production environment.

Stop application traffic and PostgreSQL 18:

```
sudo pg_ctlcluster 18 main stop
```

Install the approved previous PostgreSQL 18 package versions using explicit APT version pinning. The existing PostgreSQL 18 data directory remains the data directory unless the release notes or rollback plan require restoration from backup/snapshot.

Start PostgreSQL 18:

```
sudo pg_ctlcluster 18 main start
```

Confirm version, data directory, logs, database access and application connectivity:

```
sudo -u postgres psql -p 5432 -c "SELECT version(); SHOW data_directory;"
```

Do not treat a package downgrade as a universal rollback method; the approved rollback plan and release-specific guidance take precedence.

55. Major Upgrade vs Minor Upgrade

Area	Major Upgrade	Minor Upgrade
Example	16 → 18	18.4 → 18.6
New data directory	Yes	No
`pg_upgrade` needed	Possible	No
Dump/restore possible	Yes	Normally unnecessary
Logical replication possible	Yes	Normally unnecessary
Physical storage compatibility	Not guaranteed across majors	Maintained within major
Application testing	Mandatory	Recommended
Release-note review	Mandatory	Mandatory
Downtime	Depends on method	Usually short

56. Major Upgrade Options

PostgreSQL major upgrades can use:

```
pg_dump / pg_restore
pg_dumpall
pg_upgrade
Logical replication
Managed / third-party migration tooling
```

For PostgreSQL 16 → 18, PostgreSQL 17 is **not required as an intermediate upgrade**.

A direct:

```
16 → 18
```

upgrade is supported when the chosen method and compatibility checks allow it.

Still review PostgreSQL 17 and PostgreSQL 18 release notes for compatibility-impacting changes.

57. `pg\_upgrade` Production Checklist

Before pg_upgrade, verify:

```
Source PostgreSQL version
Target PostgreSQL version
Operating system compatibility
Old/new binary directories
Old/new configuration directories
Old/new physical data directories
Tablespaces
Disk space
Filesystem
Data checksum state
Locale
Encoding
Extensions
Shared libraries
Authentication
SSL
Archive configuration
Replication
Backup
Restore capability
Application downtime
Monitoring
Rollback plan
```

58. Additional Useful Commands

Check version:

```
SELECT version();
```

Server version:

```
SHOW server_version;
```

Data directory:

```
SHOW data_directory;
```

Checksums:

```
SHOW data_checksums;
```

WAL level:

```
SHOW wal_level;
```

Archive mode:

```
SHOW archive_mode;
```

Connections:

```
SHOW max_connections;
```

Database sizes:

```
SELECT
    datname,
    pg_size_pretty(pg_database_size(datname))
FROM pg_database
ORDER BY pg_database_size(datname) DESC;
```

Replication slots:

```
SELECT * FROM pg_replication_slots;
```

Replication status:

```
SELECT * FROM pg_stat_replication;
```

Prepared transactions:

```
SELECT * FROM pg_prepared_xacts;
```

Extensions:

```
SELECT
    extname,
    extversion
FROM pg_extension;
```

59. Backup Validation Before Upgrade

Do not stop at:

```
Backup completed successfully
```

Also verify recovery capability.

For a logical dump:

```
sudo -u postgres \
/usr/lib/postgresql/16/bin/pg_restore \
-l /DBBackups2026/PG16_PreUpgrade/dvdrental_pg16.dump \
| head
```

For a backup file checksum:

```
sha256sum -c dvdrental_pg16.dump.sha256
```

For physical backup tools, use the validation method appropriate to that tool and perform periodic restore testing.

60. Do Not Forget Replication

If PostgreSQL 16 is part of HA/replication, this standalone procedure is not enough.

The upgrade plan must cover:

- Primary
- Standbys
- Replication slots
- Archive/WAL
- Replication users
- Failover tooling
- Patroni/repmgr
- Load balancer or VIP
- Connection endpoints
- Backup
- Monitoring

Use a dedicated HA/replication upgrade runbook.

61. Do Not Delete the Old Cluster Immediately

PostgreSQL 18 starting successfully does **not** mean PostgreSQL 16 should be deleted immediately.

Wait until:

- Application testing complete
- Smoke testing complete
- Performance acceptable
- Backups working
- Restore capability confirmed
- Monitoring working
- Replication healthy
- Business acceptance complete
- Rollback period closed

Only then perform old-cluster cleanup.

62. Validated Technical Rules

Technical Area	Verified Technical Rule
Version numbering	PostgreSQL 10 and later use a two-part Major.Minor numbering model; for example, 16.15 means major 16, minor 15.
Two-part numbering	The modern two-part numbering model began with PostgreSQL 10.
pg_upgrade history	The pg_upgrade utility officially appeared with PostgreSQL 9.0.
Ubuntu binary paths	Use /usr/lib/postgresql/<version>/bin for the Ubuntu packages used in this SOP.
Ubuntu data paths	This lab uses /var/lib/postgresql/<version>/main for physical cluster data.
Ubuntu package management	Use APT package-management commands for this Ubuntu 26.04 procedure.
Ubuntu configuration layout	Cluster configuration is under /etc/postgresql/<version>/main while physical data is under /var/lib/postgresql/<version>/main.
Data checksums	For pg_upgrade, source and target checksum states must be compatible; this tested lab uses checksums OFF on both PG16 and PG18.
Configuration migration	Review and merge required PostgreSQL 16 settings into PostgreSQL 18; do not blindly overwrite the PostgreSQL 18 configuration.

Optimizer statistics	PostgreSQL 18 <code>pg_upgrade</code> transfers most optimizer statistics; missing and cumulative statistics still require post-upgrade maintenance.
Linux inode	An inode is filesystem metadata for a filesystem object; it is not the physical file itself.
LINK-mode rollback	After the new PostgreSQL 18 linked cluster starts, do not restart the original linked PostgreSQL 16 cluster.
<code>pg_control.old</code>	Restoring <code>pg_control.old</code> does not make relation files modified by PostgreSQL 18 safe for PostgreSQL 16.
Logical downgrade	A PostgreSQL 18 logical dump is not guaranteed to restore successfully into PostgreSQL 16.
Minor-version rollback	Do not assume package downgrade alone is sufficient; review release-specific actions, extensions, configuration, and backup/restore requirements.
Object inventory	<code>pg_stat_user_tables</code> and similar statistics views describe the currently connected database, not a complete cluster-wide object inventory.
Capacity planning	Do not use a universal fixed free-space number; calculate storage requirements from the actual cluster, transfer mode, WAL, temporary files, and operational headroom.
Direct major upgrade	PostgreSQL 16 can be upgraded directly to PostgreSQL 18 with a supported migration method; PostgreSQL 17 is not required as an intermediate step.

63. Production Recommendation – PostgreSQL 16 → 18

For a normal production environment where enough storage is available:

```

PostgreSQL 16
  |
  | Prechecks
  | Backup
  | pg_upgrade --check
  v
Maintenance Window
  |
  | Stop Application
  | Stop PG16 and PG18
  v
pg_upgrade COPY MODE
  |
  v
PostgreSQL 18
  |
  | Configuration Review
  | Port Cutover
  | Start PG18
  | Statistics Maintenance
  | Validation
  v
Production

```

Advantages:

```

Faster than full dump/restore
Old cluster remains independent
Immediate rollback is simpler before new PG18 writes

```

For very large databases where storage or downtime is critical, evaluate:

```

pg_upgrade --link
pg_upgrade --clone

```

```
pg_upgrade --copy-file-range
pg_upgrade --swap
Logical replication
```

with a tested rollback design.

64. Quick Reference Cheat Sheet

Major Upgrade

16 → 18

Methods:

```
pg_upgrade
pg_dump / pg_restore
pg_dumpall
Logical replication
```

Tested COPY-Mode Flow

```
PRECHECK
↓
PREPARE PG18
↓
BACKUP + BASELINE
↓
pg_upgrade --check
↓
STOP APPLICATION
↓
STOP PG16 + PG18
↓
pg_upgrade COPY
↓
CONFIGURE PG18
↓
START PG18
↓
VALIDATE
```

`pg\_upgrade --check`

```
sudo -u postgres bash -c '
cd /var/lib/postgresql/pg_upgrade_work

/usr/lib/postgresql/18/bin/pg_upgrade \
  --old-bindir=/usr/lib/postgresql/16/bin \
  --new-bindir=/usr/lib/postgresql/18/bin \
  --old-datadir=/etc/postgresql/16/main \
  --new-datadir=/etc/postgresql/18/main \
  --old-options="-D /var/lib/postgresql/16/main" \
  --new-options="-D /var/lib/postgresql/18/main" \
  --old-port=5432 \
  --new-port=5433 \
  --username=postgres \
  --check
'
```


COPY Mode

Use the same command without:

```
--check
```

and without:

```
--link
```

Link Mode

Add:

```
--link
```

Remember:

```
Once PG18 starts in link mode,
do not restart the original linked PG16 cluster.
```

PostgreSQL 18 Post-Upgrade Statistics

```
sudo -u postgres \
/usr/lib/postgresql/18/bin/vacuumdb \
--all \
--analyze-in-stages \
--missing-stats-only
```

```
sudo -u postgres \
/usr/lib/postgresql/18/bin/vacuumdb \
--all \
--analyze-only
```

Minor Upgrade

```
18.4 → 18.6
```

Typical process:

```
Read release notes
  ↓
Backup
  ↓
Stop application
  ↓
Update packages
  ↓
Start PostgreSQL
  ↓
Validate
```

65. Final DBA Takeaways

Major Version

```
16 → 18
```

requires a supported migration mechanism.

Minor Version

18.4 → 18.6

normally keeps the existing data directory.

``pg_upgrade``

Always use the **new PostgreSQL version's** ``pg_upgrade`` binary.

For this lab:

```
/usr/lib/postgresql/18/bin/pg_upgrade
```

Ubuntu Split Layout

For this lab:

```

Configuration
/etc/postgresql/<version>/main

Physical Data
/var/lib/postgresql/<version>/main

```

This distinction matters when building the `pg_upgrade` command.

COPY Mode

Uses more storage but keeps the old cluster independent.

LINK Mode

Very fast and storage-efficient, but the original PG16 linked cluster becomes unsafe to restart after PG18 starts using the shared relation files.

PostgreSQL 18 Checksums

New PostgreSQL 18 clusters enable checksums by default.

For `pg_upgrade`, match the source checksum state.

PostgreSQL 18 Statistics

PostgreSQL 18 `pg_upgrade` transfers most optimizer statistics, but missing and cumulative statistics still need post-upgrade maintenance.

Configuration

Never blindly replace PostgreSQL 18 configuration with PostgreSQL 16 configuration.

Rollback

The safe immediate rollback window for COPY mode is before PostgreSQL 18 accepts meaningful new production writes.

A production upgrade is complete only after:

```

Database Validation
+
Application Validation
+
Performance Validation
+
Backup Validation
+
Monitoring Validation
+

```

66. Official PostgreSQL References

PostgreSQL Versioning Policy

<https://www.postgresql.org/support/versioning/>

PostgreSQL 18 – Upgrading a PostgreSQL Cluster

<https://www.postgresql.org/docs/18/upgrading.html>

PostgreSQL 18 – pg_upgrade

<https://www.postgresql.org/docs/18/pgupgrade.html>

PostgreSQL 18 – initdb

<https://www.postgresql.org/docs/18/app-initdb.html>

PostgreSQL 18 – Data Checksums

<https://www.postgresql.org/docs/18/checksums.html>

PostgreSQL 18 – pg_checksums

<https://www.postgresql.org/docs/18/app-pgchecksums.html>

PostgreSQL 18 – pg_dump

<https://www.postgresql.org/docs/18/app-pgdump.html>

PostgreSQL 18 – pg_dumpall

<https://www.postgresql.org/docs/18/app-pg-dumpall.html>

PostgreSQL 18 Release Notes

<https://www.postgresql.org/docs/18/release-18.html>

PostgreSQL 18.6 Release Notes

<https://www.postgresql.org/docs/18/release-18-6.html>

67. Summary

MAJOR UPGRADE
16 → 18

Requires:
pg_upgrade / dump-restore / logical replication

TESTED LAB
Ubuntu 26.04
PostgreSQL 16.15 → 18.6
Database: dvdrental
Method: pg_upgrade
Mode: COPY

MINOR UPGRADE
18.4 → 18.6

Requires:
Release-note review

Backup
Package update
Restart / validation

COPY MODE
More storage
Old cluster remains independent
Simpler immediate rollback

LINK MODE
Very little extra storage
Very fast
Original linked PG16 unsafe after PG18 starts

POSTGRESQL 18
Checksums enabled by default for newly initialized clusters
pg_upgrade transfers most optimizer statistics